🚌 Dynamic School Bus Route Optimization & Monitoring System

A full-stack web platform that dynamically generates and monitors school bus routes based on daily attendance, bus capacity, and traffic conditions.
Node.js · Express · MongoDB · React (TypeScript) · Vite

📅 Academic Year 2025–26 | 🏛️ Computer Engineering | ⚙️ Node.js + React + MongoDB

1. PROBLEM STATEMENT & SOLUTION APPROACH

The Problem

Traditional school transportation systems rely on fixed routes and static schedules that are planned once and rarely updated. This approach fails to account for:

- Students who are absent on a given day (wasted detours, fuel, and time)
- Variable bus availability due to maintenance or driver absence
- No real-time visibility for parents on bus location or student boarding status
- Manual paperwork for attendance and route management — error-prone and slow
- No alert system to notify parents if a child doesn't board the bus

Our Solution

We built a Dynamic School Bus Route Optimization & Monitoring System that:

- Uses a Nearest Neighbor heuristic (with a pluggable Cluster Strategy) to group nearby present students onto routes efficiently
- Tracks real-time boarding events and triggers parent notifications via an Observer-pattern event system
- Provides role-specific dashboards for Admins, Drivers, and Parents using JWT-authenticated React frontends
- Stores all data in MongoDB with a clean, normalized schema and Mongoose ODM

Technology Stack

Layer	Technology	Purpose
Backend API	Node.js + Express.js (TypeScript)	REST API server
Database	MongoDB + Mongoose	Document storage & ODM
Authentication	JWT + bcrypt	Secure role-based auth
Frontend	React + Vite + TypeScript + Tailwind CSS	Interactive UI
Routing	React Router v6	SPA navigation
API Client	Axios	HTTP communication
Architecture	Layered (Controller → Service → Model)	Separation of concerns
Design Patterns	Strategy, Observer, Factory	Extensibility & decoupling

2. SYSTEM DESIGN & ARCHITECTURE

2.1 Layered Architecture

The backend follows a strict three-tier layered architecture, separating concerns across each layer:

🌐 **Client (React)**
→ Vite + TypeScript + Tailwind CSS · Role-based Dashboards

🔌 **Controller Layer**
→ Route, Bus, Attendance, Boarding, Auth, Admin, User, Tracking Controllers

⚙️ **Service Layer**
→ Business logic, Route Optimization Engine, Observer Notifications, Auth

🗃️ **Data Layer (Models)**
→ Mongoose Schemas: User, Bus, Route, Attendance, Boarding, Tracking, Alert

🍃 **MongoDB Database**
→ Document-oriented NoSQL · Mongoose ODM

2.2 Scalability & Performance Improvements

▸ **Pluggable Strategy Engine:** The RouteOptimizationEngine accepts any RouteStrategy object at runtime via setStrategy(). This means the system can swap from Nearest Neighbor to Cluster (K-Means) or even a Google Maps-backed API strategy without any controller changes — horizontally scalable algorithm space.

▸ **JWT Middleware Guard:** Auth middleware validates Bearer tokens on every protected route and extracts role from the payload. Role-based guards prevent unauthorized access at the controller level without repeated auth logic.

▸ **Singleton Observer Notifier:** The boardingEventNotifier is a singleton BoardingSubject persisted for the lifetime of the server process. New notification channels (SMS, email, push) can be added as observer implementations without touching boarding logic.

2.3 Backend Directory Structure

```
backend/src/
├── controllers/    ← HTTP request handling, delegates to services
│   ├── auth.controller.ts
│   ├── route.controller.ts      ← strategy selection lives here
│   ├── attendance.controller.ts
│   ├── boarding.controller.ts
│   └── bus.controller.ts ...
│
├── services/       ← all business logic
│   ├── routeOptimization.service.ts  ← Strategy Pattern core
│   ├── attendance.service.ts         ← triggers re-optimization
│   ├── boarding.service.ts           ← triggers Observer
│   └── auth.service.ts ...
│
├── patterns/       ← explicit design pattern implementations
│   └── observer.ts             ← Observer Pattern
│
├── factories/      ← object creation logic
│   └── user.factory.ts         ← Factory Pattern
│
└── models/         ← Mongoose schemas (7 models)
    ├── user.model.ts
    ├── bus.model.ts
    ├── route.model.ts
    ├── attendance.model.ts
    ├── boarding.model.ts
    ├── tracking.model.ts
    └── alert.model.ts
```

3. OOP CONCEPTS APPLIED

3.1 Encapsulation

Each service class encapsulates its data and behaviour, exposing only a minimal public API. Internal implementations are hidden from outside callers.

```
export class AttendanceService {
  // Private helper — not accessible from outside
  private async triggerRouteOptimization(): Promise<void> {
    // fetches students, builds inputs, calls optimization engine
  }

  // Public method hides all implementation details
  async markAttendance(studentId: string, status: 'PRESENT' | 'ABSENT') {
    // ... upserts record, then calls private method
    await this.triggerRouteOptimization();
  }
}
```

3.2 Abstraction

TypeScript interfaces define abstract contracts that concrete classes must fulfill. Consumers depend on abstractions, not implementations.

```
// Abstract contract — defines WHAT must be done, not HOW
export interface RouteStrategy {
  optimize(students: StudentInput[], buses: BusInput[]): RouteResult[];
}

// Abstract Observer contract
export interface IObserver {
  update(studentId: string, status: string): void;
}

// Abstract Subject contract
export interface ISubject {
  addObserver(observer: IObserver): void;
  removeObserver(observer: IObserver): void;
  notify(studentId: string, status: string): void;
}
```

3.3 Inheritance (via Interface Implementation / Mongoose Document)

Concrete strategy classes implement the RouteStrategy interface — a form of interface-based inheritance. Mongoose models also extend the Document interface through TypeScript generics.

```
// Both classes inherit the RouteStrategy contract
export class NearestNeighborStrategy implements RouteStrategy {
  optimize(...): RouteResult[] { /* Haversine heuristic */ }
}

export class ClusterStrategy implements RouteStrategy {
  optimize(...): RouteResult[] { /* Cluster-based grouping */ }
}
```

3.4 Polymorphism

The RouteOptimizationEngine treats both strategies polymorphically — calling the same optimize() method produces different behaviors at runtime depending on the injected strategy:

```
// RouteController selects strategy based on request param
if (strategyName === 'cluster') {
  routeOptimizationEngine.setStrategy(new ClusterStrategy());
} else {
  routeOptimizationEngine.setStrategy(new NearestNeighborStrategy());
}

// Same call — different behavior (polymorphism)
const result = routeOptimizationEngine.generateOptimizedRoute(studentInputs, busInputs);
```

Summary of OOP in codebase: Encapsulation → private fields in service classes & engine; Abstraction → RouteStrategy, IObserver, ISubject interfaces; Inheritance → interface implementation across strategies; Polymorphism → strategy swapping in RouteOptimizationEngine.

4. DESIGN PATTERNS

Pattern 1: Strategy Pattern (Behavioral)

Why it was chosen:

Route optimization is not a fixed algorithm — different school zones, fleet sizes, or operational requirements may demand different routing strategies. Using a hard-coded algorithm would make future changes require modifying the core engine. The Strategy Pattern decouples the algorithm from the engine so strategies are swappable at runtime.

Where it is implemented:

backend/src/services/routeOptimization.service.ts

Key Participants:

- Context: RouteOptimizationEngine
- Strategy Interface: RouteStrategy
- Concrete Strategy A: NearestNeighborStrategy — Haversine distance heuristic
- Concrete Strategy B: ClusterStrategy — proportional cluster grouping

```
export class RouteOptimizationEngine {
  private strategy: RouteStrategy;

  constructor(strategy = new NearestNeighborStrategy()) {
    this.strategy = strategy;
  }

  // Allow hot-swapping at runtime — Strategy Pattern core
  public setStrategy(strategy: RouteStrategy): void {
    this.strategy = strategy;
  }

  public generateOptimizedRoute(
    students: StudentInput[], buses: BusInput[]
  ): RouteResult[] {
    return this.strategy.optimize(students, buses); // Delegate to strategy
  }
}
```

Pattern 2: Observer Pattern (Behavioral)

Why it was chosen:

When a driver marks a student as boarded (or not), multiple parties may need to react: parents get notified, logs are updated, future channels (SMS gateway, push notification service) may be added. Hardcoding these calls into the boarding service would violate the Open/Closed Principle. The Observer Pattern allows new listeners to register without modifying the notification source.

Where it is implemented:

backend/src/patterns/observer.ts → triggered from backend/src/services/boarding.service.ts

Key Participants:

- Subject Interface: ISubject
- Concrete Subject: BoardingSubject
- Observer Interface: IObserver
- Concrete Observer: ParentNotifierObserver
- Singleton: boardingEventNotifier

```
// patterns/observer.ts
export class BoardingSubject implements ISubject {
  private observers: IObserver[] = [];

  addObserver(observer: IObserver): void { this.observers.push(observer); }

  notify(studentId: string, status: string): void {
    for (const observer of this.observers) observer.update(studentId, status);
  }
}

// Singleton — shared across the entire server process
export const boardingEventNotifier = new BoardingSubject();
boardingEventNotifier.addObserver(new ParentNotifierObserver());

// boarding.service.ts — triggers without knowing who's listening
boardingEventNotifier.notify(studentId, status);
```

5. SOLID PRINCIPLES

S — Single Responsibility Principle

Each class/module has exactly one reason to change.

Evidence in codebase:

- AttendanceService — handles only attendance recording and route triggering
- BoardingService — handles only boarding status and observer notification
- AuthService — handles only user registration and login
- RouteOptimizationEngine — handles only algorithm orchestration
- NearestNeighborStrategy — handles only one specific routing algorithm
- UserFactory — handles only user object creation and role validation

O — Open/Closed Principle

Classes should be open for extension but closed for modification.

Evidence in codebase:

- RouteOptimizationEngine is closed for modification — a new routing strategy is added by creating a class that implements RouteStrategy, not by changing the engine
- BoardingSubject is closed for modification — new notification channels (email, SMS) are added as new IObserver implementations and registered via addObserver()
- UserFactory can be extended with new role-specific creation logic without touching caller code

L — Liskov Substitution Principle

Objects of derived classes must be substitutable for base class objects.

Evidence in codebase:

- NearestNeighborStrategy and ClusterStrategy both implement RouteStrategy and can be substituted for each other in RouteOptimizationEngine without breaking any contract
- ParentNotifierObserver implements IObserver — any additional observer (e.g., SmsNotifierObserver) would be substitutable in the observer list

- Mongoose models all implement the Document interface and are LSP-compliant as they can substitute anywhere a Document is expected

I — Interface Segregation Principle

Clients should not be forced to depend on methods they do not use.

Evidence in codebase:

- IObserver has only update() — it doesn't expose registration or notification methods
- ISubject has only addObserver, removeObserver, and notify — nothing specific to any domain
- RouteStrategy has a single method optimize() — no extra methods that concrete strategies would be forced to ignore
- Mongoose model interfaces (IUser, IBus, IRoute ...) are segregated per collection, so each interface contains only the fields that belong to that document type

D — Dependency Inversion Principle

High-level modules should not depend on low-level modules — both should depend on abstractions.

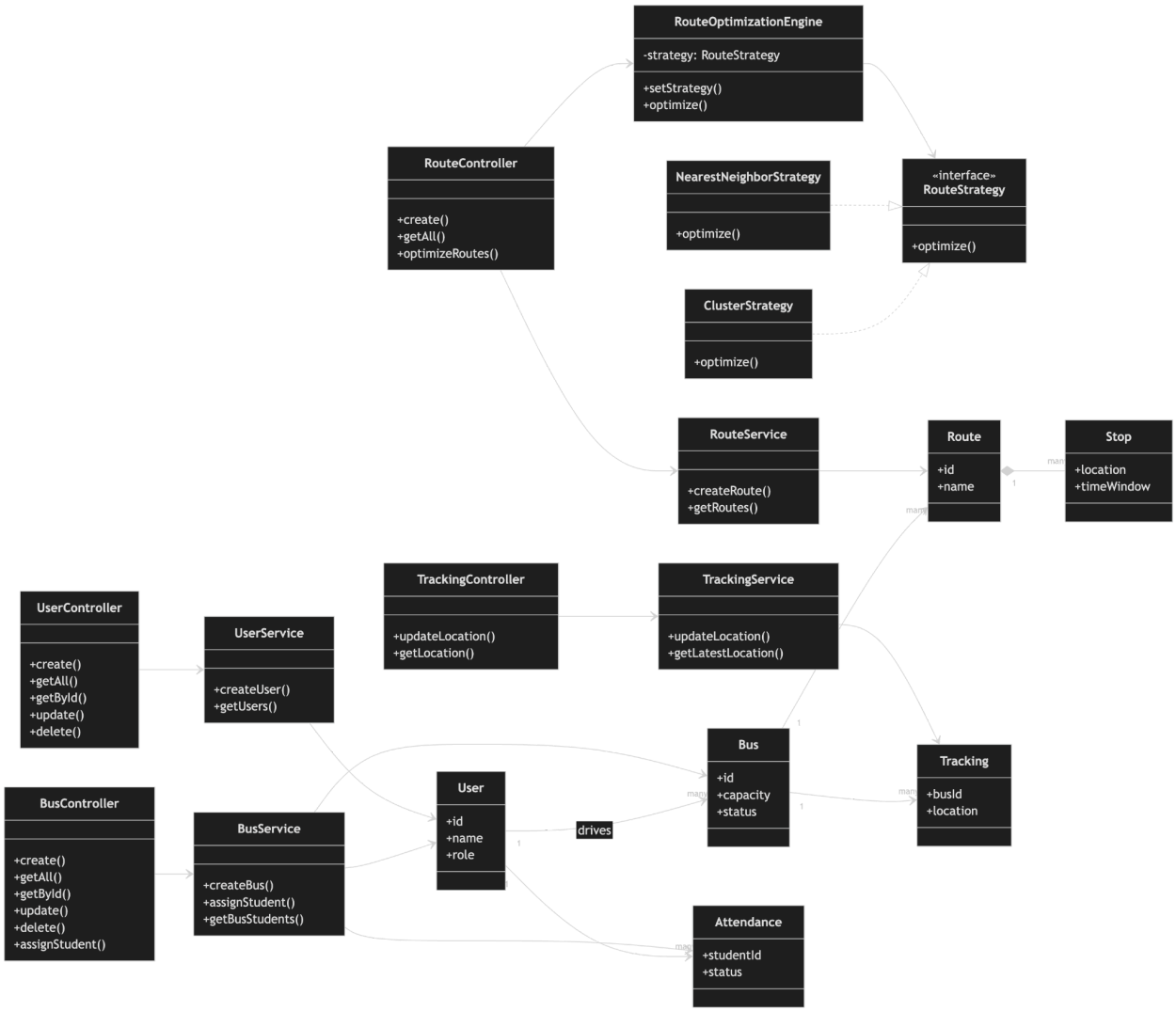
Evidence in codebase:

- RouteOptimizationEngine (high-level) depends on RouteStrategy (abstraction) — not on NearestNeighborStrategy or ClusterStrategy (concretions)
- BoardingService (high-level) depends on ISubject/boardingEventNotifier (abstraction) — adding a new observer doesn't require changing BoardingService
- Controllers depend on service modules exported as instances (already abstracted), not on Mongoose internals directly
- Auth middleware depends on the JWT abstraction (jsonwebtoken interface), not on the token format details

6. UML DIAGRAMS

6.1 Class Diagram

The class diagram captures the core domain entities and the design pattern implementations in the backend of the Dynamic School Bus Route Optimization & Monitoring System.



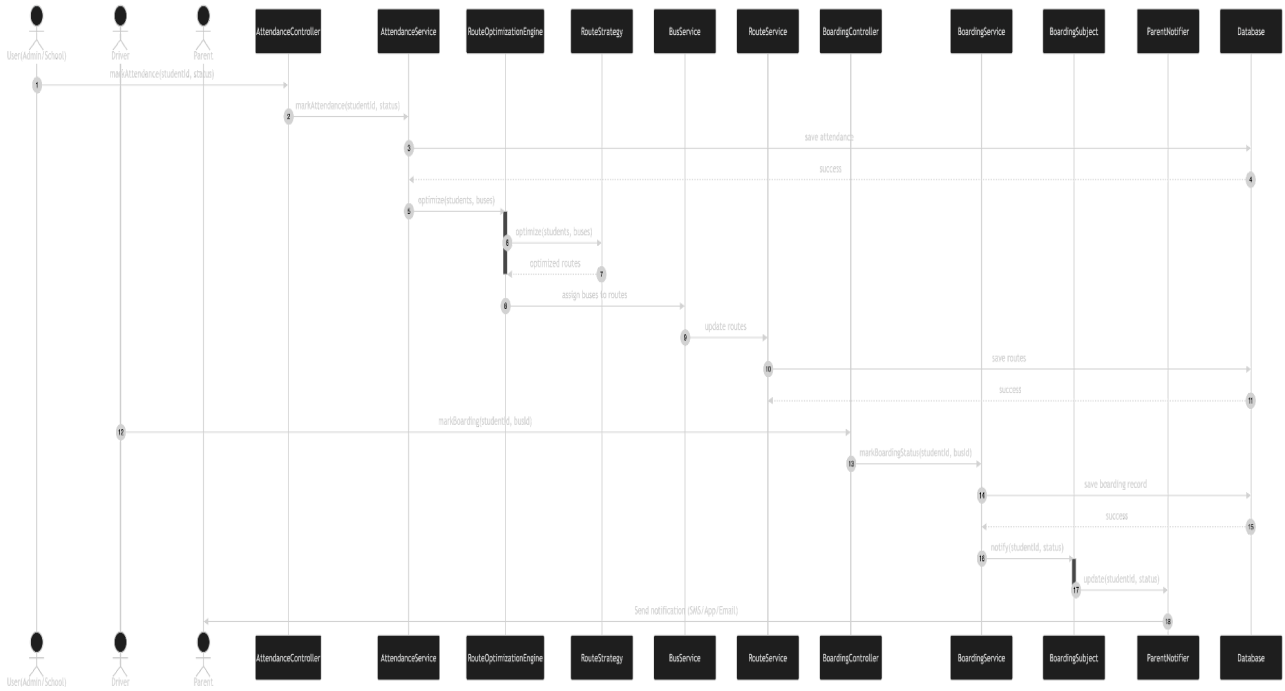
6.2 Use Case Diagram

Actors: Admin, Driver, Parent, Student

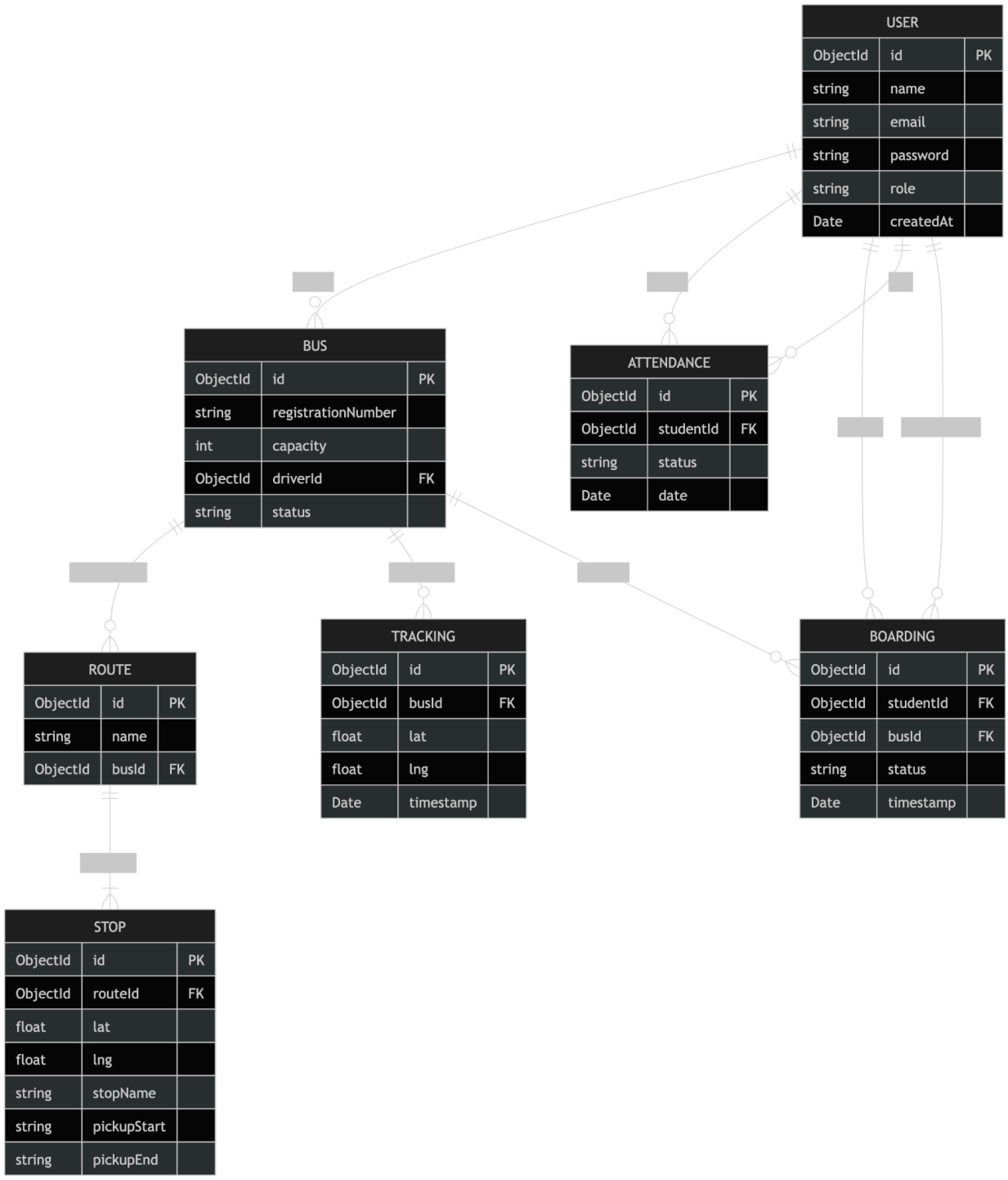
System Boundary: Dynamic School Bus System

6.3b Sequence Diagram — Boarding Notification (Observer Pattern)

Scenario: Driver marks student boarding → Parent notified.



6.4 Entity-Relationship (ER) Diagram



7. TEAM MEMBERS & CONTRIBUTION

Role	Name	Area of Contribution
Backend Lead	Rudraksh Rathod	DB models, base scaffold, server setup
Route Engine	Viraj Chafale	Optimization algorithms, route APIs, traffic sim
Monitoring & Alerts	Krishna Duble	Bus tracking, boarding logs, alert service, UML diagrams,
Frontend	Bhargav Patil	React UI, auth flows, all dashboards
Design & Docs	Kaustubh Hiwanj	Design patterns, UML diagrams, project report